

PROJECT: DIGITAL CLOCK

O.Harsha Sai Ram B.Siva Mohith

August 28, 2026

1 Contents

1.Introduction.....	2
2.Materials and their functioning.....	3
3.Building clock.....,	7
4.Software implemented.....	9
5.Digital Logics.....	17
6.Problems Faced.....	22
7.Problems rectified.....	23
8.Output.....	24

2 Introduction

The present project: digital clock is mostly based on digital electronics and arduino code programming. This project also deals with learning and implementation of digital logics and truth tables including fundamentals like k-maps. Functioning of digital clock using arduinon also includes software programming in C++ language.

3 Materials and their functioning

In this present section We are going to list down the materials used for building a six digit digital clock.

1.Arduino Uno

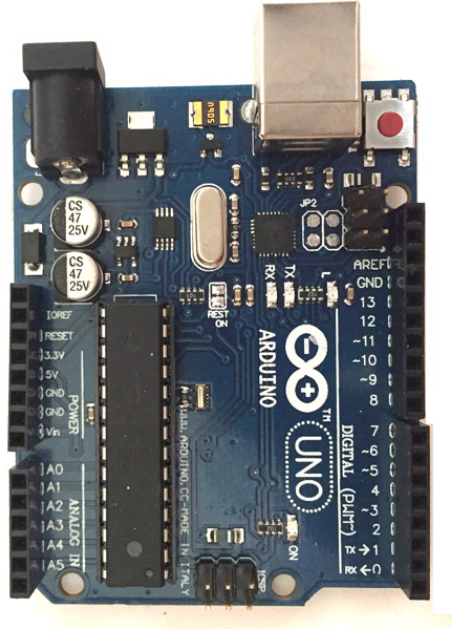


Figure 1: arduino uno

Arduino has different sockets having different functions and its the core computing part of digital clock and has various functions.

There are different group of sockets like digital sockets for giving logic impulses and power outputs for giving power to bread board and analog sockets.

arduino deals with the software embedded in the given digital clock

2. Bread Board

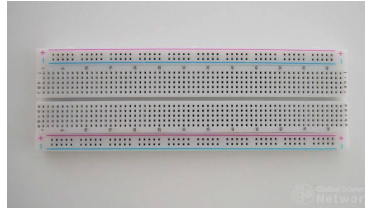


Figure 2: Bread Board

Bread board contains of many metal conducting holes. The bread board is divided into given segments, in the middle two segments the power is supplied along the column whereas the other two upper and lower rails along the row.

3. Jumper wires (male to male)

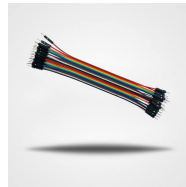


Figure 3: Jumper wires

The jumper wires are used to connect rails in bread board to arduino by inserting them into sockets.

Some of them are used to give power connection and others to give digital analogics.

4. IC 7447



Figure 4: IC 7447

This IC consists of 16 pins each having different function.

The IC is connected in the middle of bread board with 8 pins on one side and other 8 pins on other side.

5.Single core copper wires



Figure 5: Copper wires

These copper wires are used by me to do multiplexing the displays.

6.Push Buttons



Figure 6: Push buttons

I used 4 push buttons for this project those being connected to analog ports. Each being used to adjust time in the clock.

7.7 segment anode displays

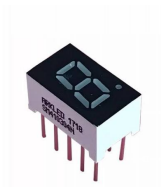


Figure 7: Anode display

This display consists of 10 pins in total and has seven segments that turn on and off as per given code.

8.USB-B to USB-A cable



Figure 8: USB-B to USB-C cable

This cable is used to connect arduino to our device to send C++ code and to give power supply.

9.Resistors



Figure 9: Resistors

The resistors are given to terminals of push buttons and sometimes used to connect ic to displays.

4 Building Clock

This guide will take you step-by-step through setting up the hardware using your specific components and then provide you with the software to make it run.

Preparation: Understanding Your Components

Before we start wiring, it's essential to visualize how our key components interact.

First, let's establish a clear image of the core setup and the components we are using. We have the Arduino Uno, a standard half-size breadboard, six Common Anode Seven-Segment Displays (identified by their 'a' through 'g' pins and common anode 'CA'), the IC 7447 BCD-to-7-Segment Decoder, and our 220 ohm (red-red-brown) and 240 ohm (red-yellow-brown) resistors.

Here is an illustrated image used for giving connections in the clock.

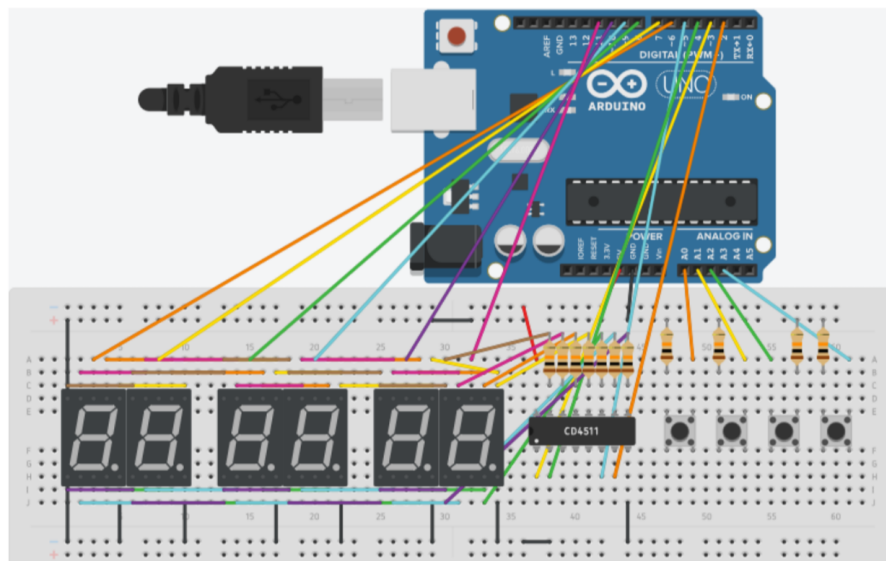


Figure 10: Hardware connections

Hardware Setup and Wiring

Now we will build the circuit shown above in layers. This project uses multiplexing to drive six displays with limited pins.

Step 1: Establish Power and Ground Rails

First, let's provide a power backbone for the entire breadboard. We will connect the Arduino's power outputs to the long vertical rails running down both sides of the breadboard.

Connect a red male-to-male wire from the Arduino 5V pin to the top positive (+) rail of the breadboard.

Connect a black male-to-male wire from the Arduino GND pin to the top negative (-) rail of the breadboard.

Use two more small jumper wires (black and red) to bridge the top power/ground rails to the bottom power/ground rails, energizing the entire board.

Step 2: Wire the 7447 Decoder IC

The 7447 IC takes a 4-bit Binary Coded Decimal (BCD) signal from the Arduino and translates it into the signals needed to light up the segments (a-g) of the displays.

Place the 7447 IC across the central ravine of the breadboard, making sure the notch faces left.

Power and Ground (IC):

- *Connect Pin 16 (VCC) of the 7447 to the 5V rail.

- *Connect Pin 8 (GND) of the 7447 to the GND rail.

*Arduino to 7447 Inputs (Data):

- *Connect Arduino Digital Pin 2 to 7447 Pin 7 (Input A).

- *Connect Arduino Digital Pin 3 to 7447 Pin 1 (Input B).

- *Connect Arduino Digital Pin 4 to 7447 Pin 2 (Input C).

- *Connect Arduino Digital Pin 5 to 7447 Pin 6 (Input D).

Step 3: Connect 7447 Outputs to the Display Segments

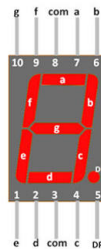


Figure 11: Pin connection of displays

This is the main data bus. We connect the output pins (a-g) of the 7447 IC to the corresponding segment pins of all six displays in parallel. When the 7447 outputs data, every display receives it. We also add the 220ohm resistors to limit current to the segments.

- *Place all six Common Anode (CA) displays side-by-side on the breadboard.

- *Segment Bus (a-g): For each display (left to right, 1 to 6):

- *Connect 7447 Pin 13 (a) through a 220ohm resistor to the display's 'a' pin.

- *Connect 7447 Pin 12 (b) through a 220ohm resistor to the display's 'b' pin.

- ...Repeat for segments c, d, e, f, g.

(Note: In the final image, these segment lines appear as the main cluster of neat, parallel blue and green wires spanning from the 7447 to the display group.)

Step 4: Wire the Common Anode (Multiplexing) Control

To make the multiplexing work, we only turn on one display at a time by connecting its Common Anode (CA) pin to 5V very quickly. We control this activation using Arduino digital pins, buffered by the 240ohm resistors.

Arduino to Display Anodes (Control):

- *Connect Arduino Digital Pin 8 through a 240ohm resistor to the CA pin of Display 1 (far left).

- *Connect Arduino Digital Pin 9 through a 240ohm resistor to the CA pin of Display 2.

- ...Repeat for Pins 10, 11, 12, and 13 to control Displays 3, 4, 5, and 6 (far right).

Step 5: Add Control Buttons for Time Setting

The final addition is the three push buttons that allow you to set the time (Hours, Minutes, and a Mode/Select button).

- *Place three push buttons on the breadboard.

- *Connect one side of all three buttons to the GND rail.

- *Connect the other side of the Hour Set button to Arduino Analog Pin A0.

- *Connect the other side of the Minute Set button to Arduino Analog Pin A1.

- *Connect the other side of the Mode/Set button to Arduino Analog Pin A2.

(Note: The internal pull-up resistors on the Arduino will be activated in code, so we don't need external resistors for these buttons.)

5 Software Implemented

The C++ code given to the clock is mentioned below:

The code given is:

```

// Shared 7447 Inputs
const int PIN_A = 0, PIN_B = 1, PIN_C = 2, PIN_D = 3;

// Enable pins for multiplexing
const int EN[] = {4,5,6,7,8,9}; // sec1, sec10, min1, min10, hr1, hr10

// Buttons
const int PAUSE_BTN = 10;
const int NEXT_BTN = 11;
const int INC_BTN = 12;
const int DEC_BTN = 13;

bool paused = false;
int lastPauseState = HIGH;
int lastNextState = HIGH;
int lastIncState = HIGH;
int lastDecState = HIGH;

// Digit selection when paused
int selectedDigit = 0; // 0..5 → sec1..hr10
unsigned long lastBlink = 0;
bool blinkOn = true;

// Current States
// Seconds
int W1=0,X1=0,Y1=0,Z1=1;
int W2=1,X2=0,Y2=1;
// Minutes
int W3=1,X3=0,Y3=0,Z3=1;
int W4=1,X4=0,Y4=1;
// Hours
int W5=1,X5=1,Y5=0,Z5=0;
int W6=0,X6=1,Y6=0;

// Timer

```

Figure 12:

```

// timer
unsigned long lastSecUpdate=0;

// Setup
void setup(){
  pinMode(PIN_A,OUTPUT); pinMode(PIN_B,OUTPUT);
  pinMode(PIN_C,OUTPUT); pinMode(PIN_D,OUTPUT);
  for(int i=0;i<6;i++){ pinMode(EN[i],OUTPUT); digitalWrite(EN[i],LOW); }
  pinMode(PAUSE_BTN,INPUT_PULLUP);
  pinMode(NEXT_BTN,INPUT_PULLUP);
  pinMode(INC_BTN,INPUT_PULLUP);
  pinMode(DEC_BTN,INPUT_PULLUP);
}

// Increment Helper
void incrementDigit(int d){
  int A,B,C,D;
  switch(d){
    case 0: { // sec ones 0–9
      A = !W1;
      B = (W1 && !X1 && !Z1)||(!W1 && X1);
      C = (!X1 && Y1)||(!W1 && Y1)||(W1 && X1 && !Y1);
      D = (!W1 && Z1)||(!W1 && X1 && Y1);
      W1=A; X1=B; Y1=C; Z1=D;
    } break;
    case 1: { // sec tens 0–5
      A = !W2;
      B = (W2 && !X2 && !Y2)||(!W2 && X2);
      C = (W2 && X2)||(!W2 && !X2 && Y2);
      W2=A; X2=B; Y2=C;
    } break;
    case 2: { // min ones 0–9
      A = !W3;
      B = (W3 && !X3 && !Z3)||(!W3 && X3);
      C = (!X3 && Y3)||(!W3 && Y3)||(!W3 && X3 && !Y3);
    }
  }
}

```

Figure 13:

```

    D = (!W3 && Z3)||!(W3 && X3 && Y3);
    W3=A; X3=B; Y3=C; Z3=D;
} break;
case 3: { // min tens 0–5
    A = !W4;
    B = (W4 && !X4 && !Y4)||(!W4 && X4);
    C = (W4 && X4)||(!W4 && !X4 && Y4);
    W4=A; X4=B; Y4=C;
} break;
case 4: { // hr ones
    if(X6==0){ // hr tens=0/1 → 0–9
        A = !W5;
        B = (W5 && !X5 && !Z5)||(!W5 && X5);
        C = (!X5 && Y5)||(!W5 && Y5)||(W5 && X5 && !Y5);
        D = (!W5 && Z5)||(W5 && X5 && Y5);
        W5=A; X5=B; Y5=C; Z5=D;
    } else { // hr tens=2 → 0–3
        A = !W5;
        B = (W5 && !X5)||(!W5 && X5);
        W5=A; X5=B; Y5=0; Z5=0;
    }
} break;
case 5: { // hr tens 0–2
    if(!(X6==0 && W6==1 && Y5==1)){
        A = !W6 && !X6;
        B = W6 && !X6;
        W6=A; X6=B; Y6=0;
    }
} break;
}
}

// Decrement Helper
void decrementDigit(int d){

```

Figure 14:

```

int A,B,C,D;
switch(d){
case 0: { // sec ones 0–9
    A = !W1;
    B = (!X1 && !W1 && ((!Z1 && Y1)|| (Z1 && !Y1))) || (!Z1 && W1 && X1);
    C = (!Z1 && Y1 && (X1||W1)) || (Z1 && !X1 && !W1 && !Y1);
    D = !X1 && !Y1 && ((Z1 && W1) || (!Z1 && !W1));
    W1=A; X1=B; Y1=C; Z1=D;
} break;
case 1: { // sec tens 0–5
    A = !W2;
    B = (Y2 && !X2 && !W2) || (!Y2 && X2 && W2);
    C = !X2 && ((Y2 && W2) || (!Y2 && !W2));
    D = 0;
    W2=A; X2=B; Y2=C;
} break;
case 2: { // min ones 0–9
    A = !W3;
    B = (!X3 && !W3 && ((!Z3 && Y3)|| (Z3 && !Y3))) || (!Z3 && W3 && X3);
    C = (!Z3 && Y3 && (X3||W3)) || (Z3 && !X3 && !W3 && !Y3);
    D = !X3 && !Y3 && ((Z3 && W3) || (!Z3 && !W3));
    W3=A; X3=B; Y3=C; Z3=D;
} break;
case 3: { // min tens 0–5
    A = !W4;
    B = (Y4 && !X4 && !W4) || (!Y4 && X4 && W4);
    C = !X4 && ((Y4 && W4) || (!Y4 && !W4));
    D = 0;
    W4=A; X4=B; Y4=C;
} break;
case 4: { // hr ones
    if(X6==0){ // hr tens=0/1
        A = !W5;
        B = (!X5 && !W5 && ((!Z5 && Y5)|| (Z5 && !Y5))) || (!Z5 && W5 && X5);
        C = (!Z5 && Y5 && (X5||W5)) || (Z5 && !X5 && !W5 && !Y5);
    }
}

```

Figure 15:

```

        C = (!Z5 && !Y5 && (X5 || W5)) || (Z5 && !X5 && !W5 && !Y5);
        D = !X5 && !Y5 && ((Z5 && W5) || (!Z5 && !W5));
        W5=A; X5=B; Y5=C; Z5=D;
    } else { // hr tens=2 → 0–3
        A = !W5;
        B = (X5 && W5) || (!X5 && !W5);
        W5=A; X5=B; Y5=0; Z5=0;
    }
} break;
case 5: { // hr tens 0–2
    if(!(X6==0 && W6==0 && Y5==1)){
        A = X6 && !W6;
        B = !X6 && !W6;
        W6=A; X6=B; Y6=0;
    }
} break;
}
}

//Main Loop
void loop(){
    // Pause toggle
    int pauseState = digitalRead(PAUSE_BTN);
    if(pauseState==LOW && lastPauseState==HIGH){
        paused=!paused;
        if(paused) selectedDigit=0;
        delay(50);
    }
    lastPauseState=pauseState;

    // Next cursor
    int nextState=digitalRead(NEXT_BTN);
    if(paused && nextState==LOW && lastNextState==HIGH){
        selectedDigit=(selectedDigit+1)%6;
        delay(50);
    }
}

```

Figure 16:

```

    }
    lastNextState=nextState;

    // Increment
    int incState=digitalRead(INC_BTN);
    if(paused && incState==LOW && lastIncState==HIGH){
        incrementDigit(selectedDigit);
        delay(50);
    }
    lastIncState=incState;

    // Decrement
    int decState=digitalRead(DEC_BTN);
    if(paused && decState==LOW && lastDecState==HIGH){
        decrementDigit(selectedDigit);
        delay(50);
    }
    lastDecState=decState;

    // Blink
    if(paused && millis()-lastBlink>=500){
        blinkOn=!blinkOn;
        lastBlink=millis();
    }

    // Display
    for(int d=0; d<6; d++){
        bool showThis=true;
        if(paused && d==selectedDigit && !blinkOn) showThis=false;
        if(showThis){
            switch(d){
                case 0: showDigit(W1,X1,Y1,Z1,EN[0]); break;
                case 1: showDigit(W2,X2,Y2,Z1,EN[1]); break;
                case 2: showDigit(W3,X3,Y3,Z3,EN[2]); break;
                case 3: showDigit(W4,X4,Y4,Z1,EN[3]); break;
                case 4: showDigit(W5,X5,Y5,Z1,EN[4]); break;
                case 5: showDigit(W6,X6,Y6,Z1,EN[5]); break;
            }
        }
    }

```

Figure 17:

```

        case 3: showDigit(W4,X4,Y4,0,EN[3]); break;
        case 4: showDigit(W5,X5,Y5,Z5,EN[4]); break;
        case 5: showDigit(W6,X6,Y6,0,EN[5]); break;
    }
    delay(1);
}
}

// Time update
if(!paused && millis()-lastSecUpdate>=1000){
    lastSecUpdate=millis();
    // Seconds Ones
    int A = !W1;
    int B = (W1 && !X1 && !Z1)||(!W1 && X1);
    int C = (!X1 && Y1)||(!W1 && Y1)||(W1 && X1 && !Y1);
    int D = (!W1 && Z1)||(W1 && X1 && Y1);
    W1=A; X1=B; Y1=C; Z1=D;
    // Seconds Tens
    if((W1|X1|Y1|Z1)==0){
        A = !W2;
        B = (W2 && !X2 && !Y2)||(!W2 && X2);
        C = (W2 && X2)||(!W2 && !X2 && Y2);
        W2=A; X2=B; Y2=C;
    }
    // Minutes Ones
    if((W1|X1|Y1|Z1|W2|X2|Y2)==0){
        A = !W3;
        B = (W3 && !X3 && !Z3)||(!W3 && X3);
        C = (!X3 && Y3)||(!W3 && Y3)||(W3 && X3 && !Y3);
        D = (!W3 && Z3)||(W3 && X3 && Y3);
        W3=A; X3=B; Y3=C; Z3=D;
    }
    // Minutes Tens
    if((W3|X3|Y3|Z3)==0){
        A = !W4;
        B = (W4 && !X4 && !Y4)||(!W4 && X4);
    }
}

```

Figure 18:

```

C = (W4 && X4)||(!W4 && !X4 && Y4);
W4=A; X4=B; Y4=C;
}
// Hours Ones
if((W3|X3|Y3|Z3|W4|X4|Y4)==0){
  if(X6==0){
    A = !W5;
    B = (W5 && !X5 && !Z5)||(!W5 && X5);
    C = (!X5 && Y5)||(!W5 && Y5)||(W5 && X5 && !Y5);
    D = (!W5 && Z5)||(W5 && X5 && Y5);
    W5=A; X5=B; Y5=C; Z5=D;
  } else {
    A = !W5;
    B = (W5 && !X5)||(!W5 && X5);
    W5=A; X5=B; Y5=0; Z5=0;
  }
// Hours Tens
if((W5|X5|Y5|Z5)==0){
  A = !W6 && !X6;
  B = W6 && !X6;
  W6=A; X6=B; Y6=0;
}
}
}
}
}
}

// Display Helper
void showDigit(int A,int B,int C,int D,int ENpin){
  digitalWrite(PIN_A,A); digitalWrite(PIN_B,B);
  digitalWrite(PIN_C,C); digitalWrite(PIN_D,D);
  for(int i=0;i<6;i++) digitalWrite(EN[i],LOW);
  digitalWrite(ENpin,HIGH);
}

```

Figure 19:

Note: We are going to send the code along with this pdf as 'clock.cpp'

6 Digital Logics

The given image gives the K-Maps and digital functions of Digital clock:

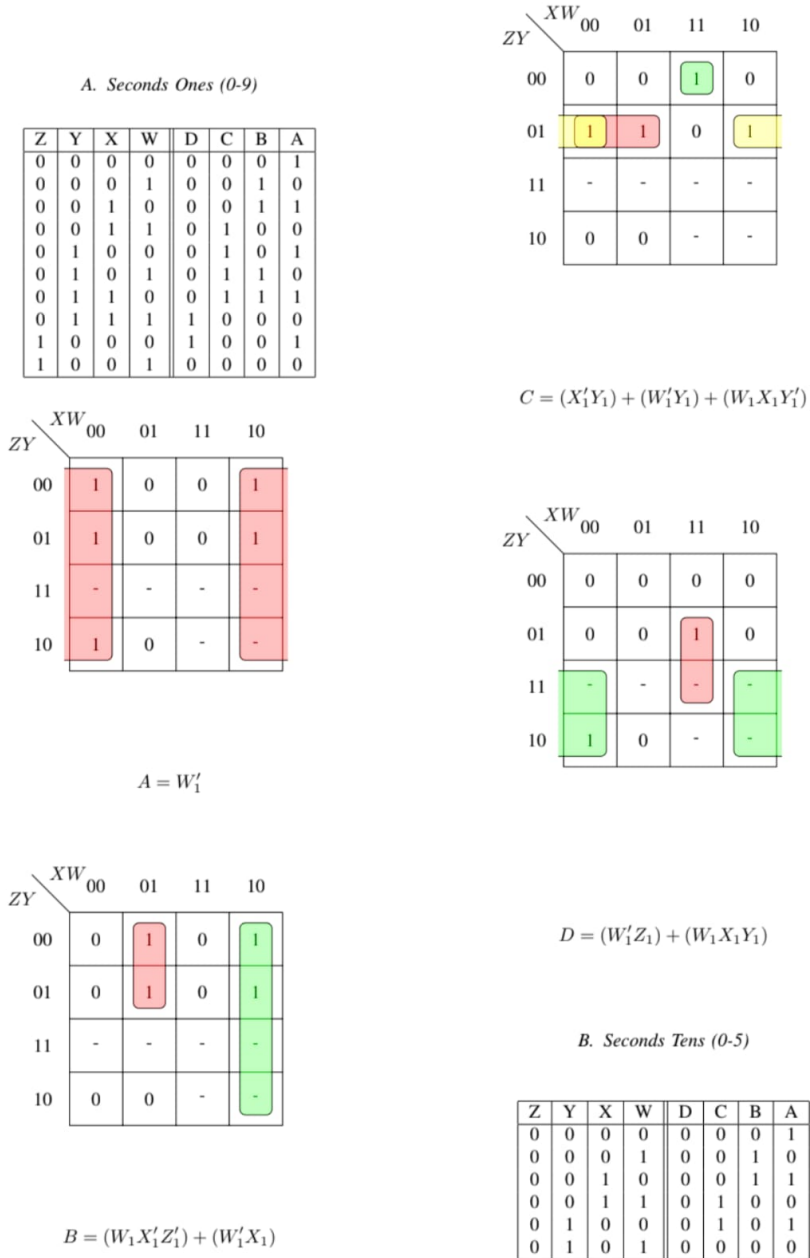


Figure 20: LOGIC 1

ZY \ XW	00	01	11	10
00	1	0	0	1
01	1	0	-	-
11	-	-	-	-
10	-	-	-	-

$$A = W'_2$$

ZY \ XW	00	01	11	10
00	0	1	0	1
01	0	0	-	-
11	-	-	-	-
10	-	-	-	-

$$B = (W_2 X'_2 Y'_2) + (W'_2 X_2)$$

ZY \ XW	00	01	11	10
00	0	0	1	0
01	1	0	-	-
11	-	-	-	-
10	-	-	-	-

$$C = (W_2 X_2) + (W'_2 X'_2 Y_2)$$

$$D = 0$$

C. Minutes Ones (0-9)

Same as Seconds Ones with W3/X3/Y3/Z3.

D. Minutes Tens (0-5)

Same as Seconds Tens with W4/X4/Y4/Z4.

E. Hours Ones

I. Tens = 0/1 → 0-9

Same as Seconds Ones with W5/X5/Y5/Z5.

II. Tens = 2 → 0-3

X	W	D	C	B	A
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1
1	1	0	0	0	0

X \ W	0	1
0	1	0
1	1	0

$$A = W'_5$$

X \ W	0	1
0	0	1
1	1	0

$$B = (W_5 X'_5) + (W'_5 X_5)$$

$$C = 0$$

$$D = 0$$

F. Hours Tens (0-2)

X	W	D	C	B	A
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	0	0	0

Figure 21: LOGIC 2

X \ W	0	1
0	1	0
1	0	-

$$A = W'_6 X'_6$$

X \ W	0	1
0	0	1
1	0	-

$$B = W_6 X'_6$$

$$C = 0$$

$$D = 0$$

VIII. DECREMENT LOGIC

A. Seconds Ones (0-9)

Z	Y	X	W	D	C	B	A
0	0	0	0	1	0	0	1
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	1
0	0	1	1	0	0	1	0
0	1	0	0	0	0	1	1
0	1	0	1	0	1	0	0
0	1	1	0	0	1	0	1
0	1	1	1	0	1	1	0
1	0	0	0	0	1	1	1
1	0	0	1	1	0	0	0

ZY \ XW	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	-	-	-	-
10	1	0	-	-

$$A = W'_1$$

ZY \ XW	00	01	11	10
00	0	0	1	0
01	1	0	1	0
11	-	-	-	-
10	1	0	-	-

$$B = (X'_1 W'_1 ((Z'_1 Y_1) + (Z_1 Y'_1))) + (Z'_1 W_1 X_1)$$

ZY \ XW	00	01	11	10
00	0	0	0	0
01	0	1	1	1
11	-	-	-	-
10	1	0	-	-

$$C = (Z'_1 Y_1 (X_1 + W_1)) + (Z_1 X'_1 W'_1 Y'_1)$$

Figure 22: LOGIC 3

ZY	XW			
	00	01	11	10
00	1	0	0	0
01	0	0	0	0
11	-	-	-	-
10	0	1	-	-

ZY	XW			
	00	01	11	10
00	0	0	1	0
01	1	0	-	-
11	-	-	-	-
10	-	-	-	-

$$B = (Y_2 X_2' W_2') + (Y_2' X_2 W_2)$$

$$D = X_1' Y_1' ((Z_1 W_1) + (Z_1' W_1'))$$

B. Seconds Tens (0-5)

Z	Y	X	W	D	C	B	A
0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	1
0	0	1	1	0	0	1	0
0	1	0	0	0	0	1	1
0	1	0	1	0	1	0	0

ZY	XW			
	00	01	11	10
00	1	0	0	0
01	0	1	-	-
11	-	-	-	-
10	-	-	-	-

$$C = X_2' ((Y_2 W_2) + (Y_2' W_2'))$$

$$D = 0$$

C. Minutes Ones (0-9)

Same as Seconds Ones with W3/X3/Y3/Z3.

D. Minutes Tens (0-5)

Same as Seconds Tens with W4/X4/Y4/Z4.

E. Hours Ones

I. Tens = 0/1 → 0-9

Same as Seconds Ones with W5/X5/Y5/Z5.

II. Tens = 2 → 0-3

X	W	D	C	B	A
0	0	0	0	1	1
0	1	0	0	0	0
1	0	0	0	0	1
1	1	0	0	1	0

$$A = W_2'$$

ZY	XW			
	00	01	11	10
00	1	0	0	1
01	1	0	-	-
11	-	-	-	-
10	-	-	-	-

Figure 23: LOGIC 4

		<i>W</i>	
		0	1
<i>X</i>	0	1	0
	1	1	0

$$A = W'_5$$

		<i>W</i>	
		0	1
<i>X</i>	0	1	0
	1	0	1

$$B = (X_5W_5) + (X'_5W'_5)$$

$$C = 0$$

$$D = 0$$

F. Hours Tens (0-2)

X	W	D	C	B	A
0	0	0	0	1	0
0	1	0	0	0	0
1	0	0	0	0	1

		<i>W</i>	
		0	1
<i>X</i>	0	0	0
	1	1	-

$$A = X_6W'_6$$

		<i>W</i>	
		0	1
<i>X</i>	0	1	0
	1	0	-

$$B = X'_6W'_6$$

Figure 24: LOGIC 5

7 Problems Faced

I am going list down all the problems I faced in a cronological order:

1.LED problem

After making all the connections from arduino to IC and bread board excluding push buttons.

One LED is not glowing and other having two lines missing and the other one is cathode.

2.Bread Board problem 1

After correcting LEDs issue, while multiplexing we got to notice that four pins are already stuck in the bread board.

We noticed that a power rail is not functioning after doing multiplexing and arduino connections.

3.Arduino problem

After solving the bread board problem we made all the other connections including buttons.

But when we gave power supply only four displays glowed and showed '4' and after few minutes it showed '1'.

4.Bread board problem 2

After solving the arduino problem, I gave all the connections and gave the C++ code to arduino but then it showed '8's instead of '0's.

I thought it was IC problem and even changed it but i didn't work.

Then I tried to check the connections and change them but it didn't work.

8 Problems Rectified

We are going to list down all the solutions for my problems listed:

1.LED solution

While the LEDs are having the problem listed we used a single cathode/anode display setup to check whether each device is working or not....

Then by performing the test we identified them and changed them from lab.

2.Bread board solution 1 After leaving the rows having pins stuck we made connections for all the other but then we ran out of space and there was a slot left for multiplexing.

Then we again went to lab and changed it.

3.Arduino solution After changing bread board and giving all the connections we came to understand that 4 of the displays are glowing '4' and others are not glowing then we identified that there was some other code in arduino. When We tried to install new code its coming that its having error then we tried from other devices and finally changed arduino.

4.Bread board solution 2 After giving all the connections from arduino to breadboard and after giving a fully functional code.

Then We thought there was error in the connections and changed all the connections.

even after everything The '0's looked as '8's then We changed the breadboard and made all the connections once again and it did work.....

9 Output

Now we have a fully functional digital clock as shown below:

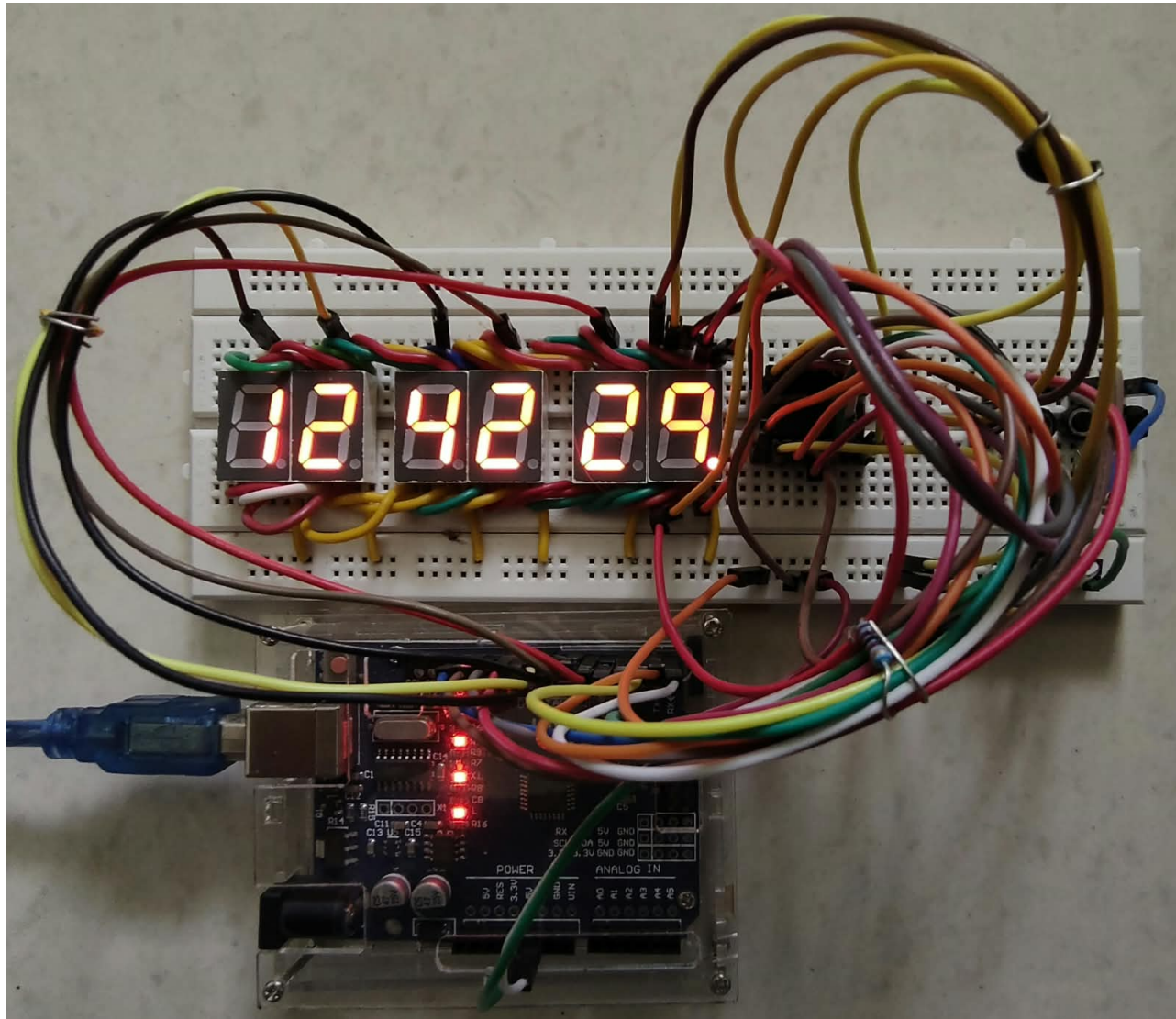


Figure 25: Final Output

It has four buttons first one for edit mode and next one for changing to edit the display next for increment digit and next for decrement digit.